EEE8087 Time-Slicing RTOS User Manual

Sachin Balappa Athani

Submission Date: 16th January 2026

1. General Description

This document is a user manual for a simple time-slicing real-time operating system (RTOS) implemented for the EEE8087 coursework. It is intended for application programmers who wish to develop multitasking programs on top of the provided runtime system. The manual focuses on how the system behaves, what services it provides, and how those services should be used correctly. It is not a detailed description of the internal implementation.

The RTOS allows multiple independent tasks to execute concurrently on a single processor by means of **time slicing**. Each ready task is allocated a fixed time slice, and when this interval expires, the scheduler suspends the current task and resumes the next ready task in rotation. This rapid switching gives the appearance of parallel execution, allowing application programs to be structured as several independent tasks rather than a single monolithic program. Understanding time-slicing is essential for application developers because it affects task timing, execution order, and the need for synchronization when accessing shared data.

Tasks are independent sequential programs with their own register sets and stack areas. The RTOS ensures that each task resumes execution exactly where it left off after being preempted. The scheduling is **round-robin**, giving all tasks fair access to the processor without priorities. Programmers can write tasks as if they have exclusive access to the CPU while the RTOS transparently handles context switching and task rotation.

The system provides a small set of system calls that tasks invoke via software interrupts. These services allow tasks to create and delete other tasks, wait for a specified number of time slices, and coordinate access to shared resources using a mutex. Execution times of system calls are bounded and predictable, making the system behaviour deterministic and suitable for small embedded applications and educational use.

Memory for each task is separated, and each task is allocated a dedicated stack to prevent interference. When creating tasks, the programmer specifies the code start address and top-of-stack address. Proper stack allocation is critical because overlapping memory regions can cause unpredictable behaviour.

On system startup, the RTOS initializes its internal structures and automatically creates a single default task (T0), which typically performs system-wide initialization, such as creating additional tasks, initializing shared variables, and setting up the mutex. T0 continues to execute concurrently with other tasks, ensuring the system always has at least one active task.

Overall, the RTOS combines **multitasking, time-slicing, and a simple system call interface** to provide a predictable, deterministic, and user-friendly environment for developing small concurrent applications.

2. Task Model and Execution Behaviour

2.1 Tasks and Concurrency

A task is an independent sequential program with its own register set and stack. Each task is written as if it has exclusive access to the processor. The RTOS is responsible for saving and restoring task context when switching between tasks.

All tasks are treated equally by the scheduler. The system uses a round-robin scheduling policy in which each ready task is given a fixed time slice. When a task's time slice expires, a timer interrupt causes the RTOS to save the current task's context and resume the next ready task in the list.

There is no priority mechanism in this RTOS. All ready tasks receive equal processor time, ensuring fairness but preventing preferential treatment of time-critical tasks.

2.2 Task Control Blocks (TCBs)

Each task is represented internally by a Task Control Block (TCB). A TCB contains all information required to suspend and later resume a task. Conceptually, this includes:

- Saved values of all processor registers, including the program counter (PC) and status register (SR)
- A pointer linking the TCB to the next TCB in the ready or waiting list
- A flag indicating whether the TCB slot is currently in use
- Storage for timing information when a task is waiting

The application programmer does not interact directly with TCBs. However, understanding that each task has a complete saved context helps explain why tasks can be paused and resumed transparently, and why tasks must have separate stack areas.

2.3 Task Limits

The system supports a maximum of eight tasks in total, including the initial task created at system startup (T0). This limit is fixed and determined by the number of available TCBs.

If an application attempts to create more than eight tasks, the Create Task system call fails. The system continues running without crashing, but the additional task is not created. Applications that create tasks dynamically should therefore keep track of how many tasks are active.

3. System Startup and Initial Task

When the system is reset, it performs basic initialisation and automatically creates a single initial task, known as task T0. This task starts executing at a predefined code address with a predefined stack location, and it is the first application-level code to run after system startup. The RTOS ensures that T0 is active and ready to schedule other tasks immediately.

T0 plays a similar role to the main function in a conventional program. It typically performs systemwide initialisation, such as:

• Initialising shared variables

• Initialising the mutex

• Creating additional tasks required by the application

After creating other tasks, T0 continues to run concurrently with them and participates in normal timesliced scheduling. The system does not allow T0 to be deleted, ensuring that at least one task is always available to run and that the system remains operational even if other tasks terminate. By maintaining T0, the system guarantees continuous task execution and stable multitasking behaviour.

4. Memory Organisation and Stacks

Each task must be assigned its own stack region in memory. The stack is used to store local variables, return addresses, and saved processor state during interrupts and system calls. All stacks grow downwards in memory, and each task operates exclusively within its own stack region. Separate stacks ensure that tasks do not overwrite each other's data during time-sliced execution.

When creating a task, the application programmer must specify:

• The start address of the task's code

• The top (highest address) of the task's stack

It is essential that stack regions for different tasks do not overlap. Overlapping stacks can lead to unpredictable behaviour, data corruption, and system crashes, and are among the most difficult errors to identify and diagnose in a multitasking environment. As a guideline, allocating at least 1 KB of stack space per task is recommended, with larger allocations used for more complex tasks or tasks that use many local variables and deeply nested subroutine calls. Proper stack allocation directly supports reliable multitasking and predictable time-slicing behaviour.

5. System Call Interface

All RTOS services are accessed through a software interrupt. To invoke a system call, a task loads a function identifier into register D0, places any required parameters in registers D1, D2, and so on, and executes the trap instruction. This uniform calling convention makes system calls easy to use and consistent across the API, allowing programmers to reliably interact with the RTOS without needing to understand internal task management or context switching.

Each system call has a clearly defined purpose, parameters, and execution time, making it predictable. Tasks can be created, deleted, or suspended using these calls, and shared resources can be safely protected. The consistent interface reduces programmer errors and supports structured multitasking design.

5.1 Create Task (Function ID = 1)

Purpose:

Creates a new task and adds it to the set of ready tasks.

Parameters:

- D0 = 1 (function identifier)
- D1 = Start address of the task's code
- D2 = Top address of the task's stack **Behaviour:**

If a free TCB is available, the RTOS creates a new task with a clean register state and adds it to the ready list. The calling task continues executing immediately; the new task begins execution when the scheduler next selects it.

If no free TCB is available, the call fails and the system continues running without creating the task.

Execution time: Approximately 45–60 instructions.

5.2 Delete Task (Function ID = 2)

Purpose:

Terminates the calling task and frees its resources.

Parameters:

- D0 = 2 **Behaviour:**

The calling task is removed from the ready list and its TCB slot is marked as unused. Control never returns to the deleted task.

The system does not allow deletion of the initial task (T0). Attempts to delete T0 are ignored, and the task continues running.

Execution time: Approximately 30–40 instructions.

5.3 Wait Time (Function ID = 6)

Purpose:

Suspends the calling task for a specified number of time slices.

Parameters:

- D0 = 6
- D1 = Number of time slices to wait (each slice = 100 ms) **Behaviour:**

The task is moved from the ready list to a waiting list and does not consume processor time while waiting. When the specified time has elapsed, the task is returned to the ready list.

Multiple tasks may wait simultaneously. If two tasks' waiting periods expire at the same time, they are returned to the ready list in the order they were waiting.

If the calling task is the only task in the system, the wait is skipped to avoid deadlock.

Execution time: Approximately 40–55 instructions.

5.4 Initialise Mutex (Function ID = 5)

Purpose:

Initialises the global mutex to a locked or unlocked state.

Parameters:

- D0 = 5
- D1 = Initial value (0 = locked, 1 = unlocked) **Behaviour:**

Sets the mutex to the specified value. This call does not wake waiting tasks.

Execution time: Approximately 15–20 instructions.

5.5 Wait Mutex (Function ID = 3)

Purpose:

Acquires exclusive access to shared resources.

Parameters:

- D0 = 3 **Behaviour:**

If the mutex is available, the task continues immediately and the mutex becomes locked. If the mutex is already locked, the task blocks until it is signalled.

Multiple tasks may wait on the mutex simultaneously. Tasks are awakened in first-in, first-out order.

If the calling task is the only task in the system, the wait is skipped to prevent deadlock.

Execution time: Approximately 25–50 instructions, depending on whether blocking occurs.

5.6 Signal Mutex (Function ID = 4)

Purpose:

Releases the mutex and potentially wakes a waiting task.

Parameters:

- D0 = 4 **Behaviour:**

If one or more tasks are waiting on the mutex, the first waiting task is made ready and acquires the mutex. If no tasks are waiting, the mutex becomes unlocked.

Execution time: Approximately 35–45 instructions.

6. Shared Data and Mutual Exclusion

When multiple tasks access shared data, race conditions can occur if updates are not protected. The mutex provided by the RTOS ensures that only one task at a time can enter a critical section, preventing inconsistent or corrupted data values. This protection is essential in applications where multiple tasks may concurrently update counters, flags, or other shared state.

Only data that is modified using read–modify–write sequences requires mutex protection. Shared variables that are only written by one task and read by others can often be accessed safely without a mutex. It is essential to always pair a Wait Mutex call with a corresponding Signal Mutex call. Failing to do so can result in permanent deadlock, leaving tasks blocked indefinitely and potentially freezing part of the application. Correct use of the mutex ensures reliable task coordination even under heavy multitasking conditions.

7. Test Programs

Two test programs are provided to demonstrate correct use of the RTOS services and to validate that the system behaves as specified under realistic multitasking conditions. These programs are not intended

as performance benchmarks, but as illustrative examples that show how tasks interact, how system calls are used, and how common concurrency problems are handled.

7.1 Stopwatch Program

The stopwatch program demonstrates basic task creation, concurrent execution, and the use of timerbased delays. It implements a simple two-digit stopwatch that can be started and stopped using a pushbutton input.

The application consists of two tasks executing concurrently. One task is responsible for maintaining and updating the displayed time value, while the second task continuously monitors the pushbutton state. When the button is pressed, the monitoring task toggles a shared control variable that indicates whether the stopwatch is running or paused.

The timing task uses the *Wait Time* system call to delay execution by one second between updates. This ensures that the display increments at a consistent rate without consuming processor time unnecessarily. By suspending itself during the delay, the task allows other ready tasks to execute, demonstrating efficient processor sharing through time slicing rather than busy waiting.

This program illustrates that tasks can operate independently while still coordinating through shared state. In this case, the shared control variable does not require mutex protection because one task only writes to it and the other only reads it. Since no read–modify–write sequence occurs, no race condition is possible. This example highlights that mutexes are required only when concurrent modification of shared data is possible, and not for all shared variables.

Overall, the stopwatch program provides a clear demonstration of task creation, periodic task execution, and cooperative interaction between tasks using simple shared data.

7.2 Radiation Monitor Program

The radiation monitor program demonstrates correct use of the mutex mechanism to protect shared data accessed by multiple tasks. It models a scenario in which two independent detector tasks continuously record events and contribute to a shared total count.

In this program, two tasks simulate independent event counters, each maintaining its own local count while also updating a shared global counter. A third task periodically delays for a fixed time interval and then reads all counters to calculate and display summary results.

The shared counter is accessed by both detector tasks using a read–modify–write sequence. Without protection, this would result in race conditions and lost updates when task execution is interrupted during the update sequence. To prevent this, each detector task acquires the mutex before updating the shared counter and releases it immediately afterwards. This guarantees exclusive access and ensures that each increment is correctly applied.

After the measurement interval expires, the monitoring task compares the sum of the two local counters with the shared counter value. Under correct operation, the shared counter closely matches the sum of the local counters, with only minimal discrepancy possible due to task scheduling boundaries. This verification step provides clear evidence that the mutex mechanism prevents data corruption even when tasks execute concurrently at high rates.

This program demonstrates the importance of mutual exclusion in multitasking systems and confirms that the mutex implementation supports repeated acquisition and release without deadlock or starvation.

It also shows that multiple tasks can safely coordinate using RTOS services while maintaining predictable and correct system behaviour.

8. Conclusion

This Real-Time Operating System provides a simple, predictable multitasking environment based on time slicing and round-robin scheduling. Through a small and clearly defined set of system calls, application programmers are able to create concurrent tasks, introduce controlled timing delays, and safely share data between tasks using mutual exclusion. The consistent system call interface and documented execution times make the system easy to understand and practical to use in real applications.

By deliberately limiting complexity, the system avoids many of the pitfalls commonly associated with multitasking software. The absence of task priorities and the use of fair scheduling ensure that all tasks receive regular processor time, resulting in deterministic and easily analysable behaviour. This predictability is particularly valuable in embedded systems where understanding worst-case execution timing is often more important than achieving maximum throughput.

The inclusion of a global mutex provides essential protection against race conditions when tasks access shared data. The accompanying test programs demonstrate that correct use of this mechanism prevents data corruption even under heavy contention, while also illustrating when mutex protection is not required. This helps programmers apply synchronisation techniques appropriately rather than unnecessarily.

Overall, the RTOS is well suited to educational use and to small embedded applications where clarity, correctness, and determinism are prioritised over advanced scheduling features. It provides a solid foundation for understanding real-time multitasking concepts and offers a reliable platform on which simple concurrent applications can be developed with confidence.